TECHNICAL UNIVERSITY OF MUNICH

School of Computation, Information and Technology
Informatics

Master's Thesis in Informatics

DRAFT - Extension of Pacing Diagrams and Pacing Analysis in pix:e

Rai Trouvain

TECHNICAL UNIVERSITY OF MUNICH

School of Computation, Information and Technology
Informatics

Master's Thesis in Informatics

DRAFT - Extension of Pacing Diagrams and Pacing Analysis in pix:e

Thesis Proposal: Irgendetwas mit Usability in PIX:E

Author:	Rai Trouvain
Supervisor:	Prof. Dr. Georg Groh
Advisors:	Julian Geheeb M.Sc.
Submission Date:	DRAFT - 04.08.2026

Disclaimer

I confirm that this Master's thesis is my own work and I have documented all sources and material used.

(Date)

(Rai Trouvain)

Acknowledgements

The acknowledgements

Abstract

English Abstract

Zusammenfassung

German Abstract

Contents

1 Introduction

2 Background

2.1 Game Design Tools??

✏️

2.2 Player Experience

✏️

2.3 PaceMaker

(Geheeb, Dyrda, and Geheeb 2024)

- PaceMaker: predecessor of PxCharts

"The toolkit, PaceMaker, allows the user to design a non-linear experience chart and subsequently plot relevant information like intensity or gameplay category of each node along a path on the chart."

-> presented as prototype/PoC

"Pacing describes the rhythm that results from the recurring patterns of rhythmic parameters in time. Rhythmic parameters are divided into artifact and experience parameters."

-> mostly numerical or categorical

2.3.1 Functionalities and Evaluation of PaceMaker

1. statecharts for modeling (Harel 1987)
   - nodes = beats
2. experience specification: properties that can be assigned to a beat
   - name, description, narrative/gameplay/overall intensity, gameplay category, expected playtime
   - in p:xe: PxComponents, can be defined by user (available datatypes: TODO)
3. path selection
   - start/intermediate/end beats -> Dijkstra
   - path snapshots
   - implementation in p:xe part of this work
4. pacing diagrams
   - visualize intensity or gameplay category per beat
   - in p:xe: selection based on PxComponents
     - comparison of different properties per beat

    - comparison of different paths
- usability issues
- nesting/concurrency

2.4 Lock-and-Key Puzzles

2.4.1 Definition

(Ashmore and Nitsche 2007): “The puzzle is finding out what is an obstacle, what and where is a key to overcome it, and finally using the key to master the challenge.”

(Ashmore and Nitsche 2007): “Obstacles may not be passed until the player obtains some token (such as an item or skill)”

may be literal locks and keys, but may also be other items (e.g. weapons) or skills (e.g. double jump).

✏️ add more content

2.4.2 Taxonomy for Locks and Keys

by (Dormans 2017)

- overview of different aspects/types of locks and keys that may appear in puzzles

✏️ add context on dormans?

2.4.2.1 Locks

(Dormans 2017): “When you unlock a door, that door might remain unlocked forever (permanent), for a short period of time (temporary), or until it is relocked (reversible). Sometimes, a lock collapses after use, allowing the player only to pass once.”

-> locks: **permanent, temporary, reversible, collapsible**

(Dormans 2017): “Certain locks allow you to cross only in one direction (valves), while others can only be opened from one direction but traversed in two directions after they are opened (asymmetrical). […] Valves do not always require a key.”“

-> locks: **valve, asymmetrical**

(Dormans 2017): “A safe lock is guaranteed to have a solution, while an unsafe lock is not.”

-> locks: **safe or unsafe**

(Dormans 2017): “Some locks are barriers that might be navigated without a key, but this crossing the barrier might be uncertain or impose a certain risk.”

2.4.2.2 Keys

(Dormans 2017): "Single-purpose keys can only be used to open a lock, and for nothing else, while multipurpose keys can also be used in different ways."

-> keys: **single-purpose or multi-purpose**

(Dormans 2017): "Particular keys are the only thing that unlocks a particular lock, whereas several nonparticular keys might unlock a single lock."

-> keys: **particular or non-particular**

(Dormans 2017): "Keys that are destroyed somehow in the process of unlocking a door are consumable, while keys that are not are persistent."

-> keys: **consumable or persistent**

(Dormans 2017): "Levers and switches are the best example of keys that are fixed in place (and typically single purpose and particular as well)."

-> keys: **fixed or not**

3 Path-Based Diagrams

✏️ potentially refactor: 1 chapter for implementation, provide context (tech stack etc)

- basic idea already present in PaceMaker: select path in statechart and visualize data along path
- main design guideline: build upon high configurability already present in the system
- following sections describe functionalities, implementation choices and wokflow/ usage of these features implemented in pixe

3.1 Path Selection

3.1.1 Functionality

- selection of nodes in statechart triggers calculation of path connecting them
- selected path is highlighted (visual feedback)
- path is subsequently used for diagram generation

3.1.2 Implementation

- select start + end node, optionally intermediate nodes -> calculate path using dijkstra
- `pathsApi.ts`: to allow for different algorithms in the future (e.g. specific pathfinding for nestedness and concurrency)

✏️ add high-level flowchart for path calculation logic (from node selection to highlighting)

3.1.3 Workflow

- selection of at least two nodes
  - Ctrl + click
- path is calculated along nodes **in order of selection**
- if path is found, nodes along the path are highlighted in purple. if not, start/end nodes are highlighted in red.
- path can be de-selected by removing node selection (clicking anywhere)

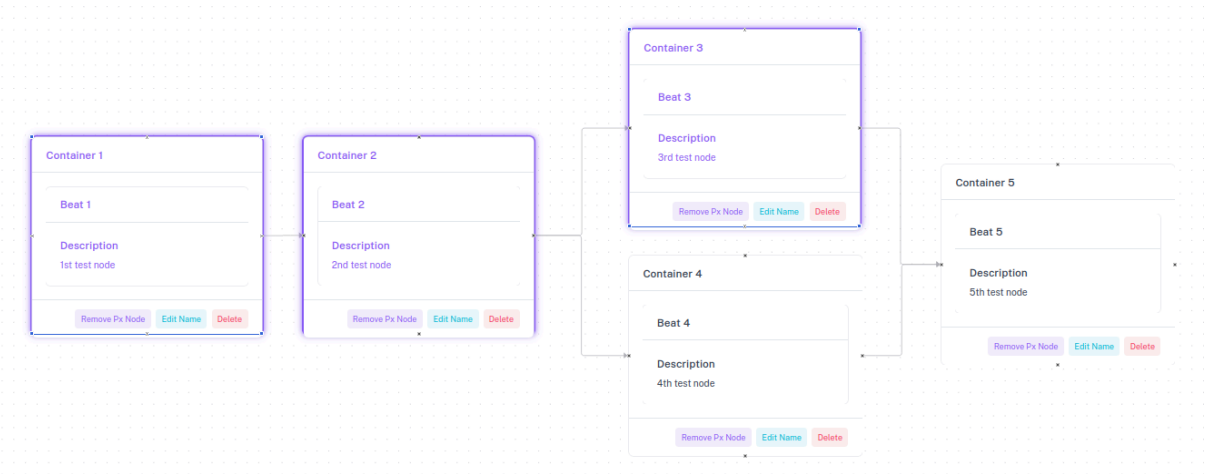

Figure 1: Path highlighted in state chart

3.1.4 Limitations

- calculation fully done in frontend
- paths are not persisted
- no visual indicator for start/end nodes

3.2 Diagram Generation

3.2.1 Functionality

- expandable element on statechart page
  - usability: users can hide diagrams while working on statechart
- can create multiple diagrams and remove diagrams (red because diagrams do not persist)

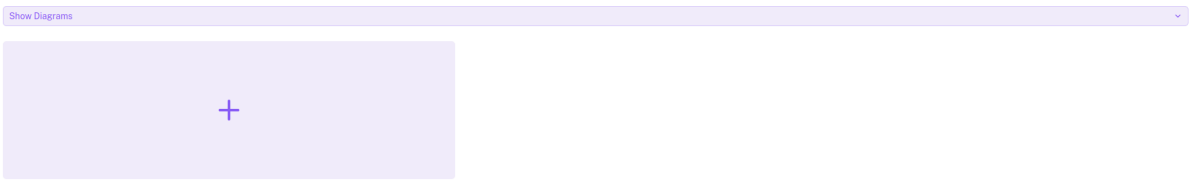

Figure 2: Upper section of the charts page with expandable element and UI for diagram creation

- selection of one or multiple component definitions for the y axis
- optionally selection of one component for the x axis (e.g. estimated playtime)
  - values are summed up along selected path
  - default: equal spacing

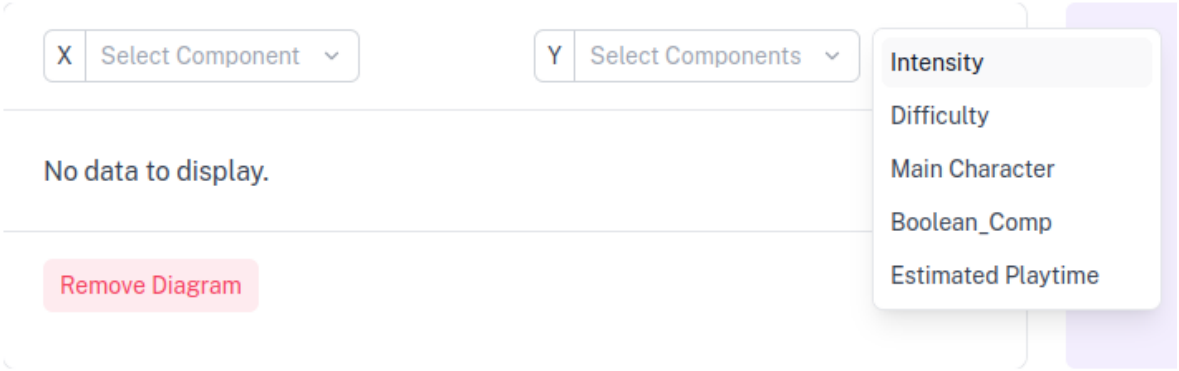

Figure 3: Selectors for components to be mapped to X/Y axes

- selected path along x axis
  - if no path selected: all nodes, in no specific order

Figure 4: Diagram visualizing component along selected path

Figure 5: Diagram visualizing component across all nodes

3.2.2 Implementation

- chartJS library
- modular architecture: `PxDiagrams` as wrapper element
  - manages diagrams
    - currently: passes path (reactively), deletes diagrams
    - future work: path snapshots, different data types
- data is extracted from nodes once per diagram, switching between axis configurations is then done via different parsing configurations

3.2.3 Workflow

1. add new diagram
2. in any order:
   - select one or no component for x axis
   - select one or more component for y axis
   - select path
- changes in any of the configurations will be reflected live in the diagram

3.2.4 Limitations

- diagrams do not persist (reset when reloading page)
- only line diagrams for now
- same path selection for all diagrams

4 Locks and Keys

- novel functionality (not in PaceMaker)
- can be used both low-level (e.g. to model individual puzzles/dungeons) or high-level (e.g. to model storylines and their prerequisites) -> some range in use cases. possible high complexity in lock/key type puzzles ✏️ add citation , so analysis/verification/debugging functionality has potential for high impact.

4.1 Requirements

- use 2.4.2 (Taxonomy for Locks and Keys) as reference for requirements to the lock/key functionality in pix:e
- in statechart: conditional transition
- reflect concept of keys as abstract tokens, different kinds to distinguish: keys/locks should have some category to specify this
- locks: **permanent, temporary, reversible, collapsible**
  - attribute `unlockMode` on locks, can then be referred to in pathfinding logic
- keys: **single-purpose or multi-purpose**
  - attribute `type` on keys: `ability`, `item`
- keys: **particular or non-particular**
  - probably easier to understand when this is a property of a lock, i.e. key requirement
- keys: **consumable or persistent**
  - attribute `consumable` on keys, can then be referred to in pathfinding logic
- keys: **fixed or not**
  - attribute `fixed` on keys, can then be referred to in pathfinding logic
- locks: **valve, asymmetrical**
  - modeled via bidirectional edges with locks vs two uni-directional edges, one with lock and one without
- locks: **safe or unsafe**: not currently modeled

4.2 Resulting Data Model

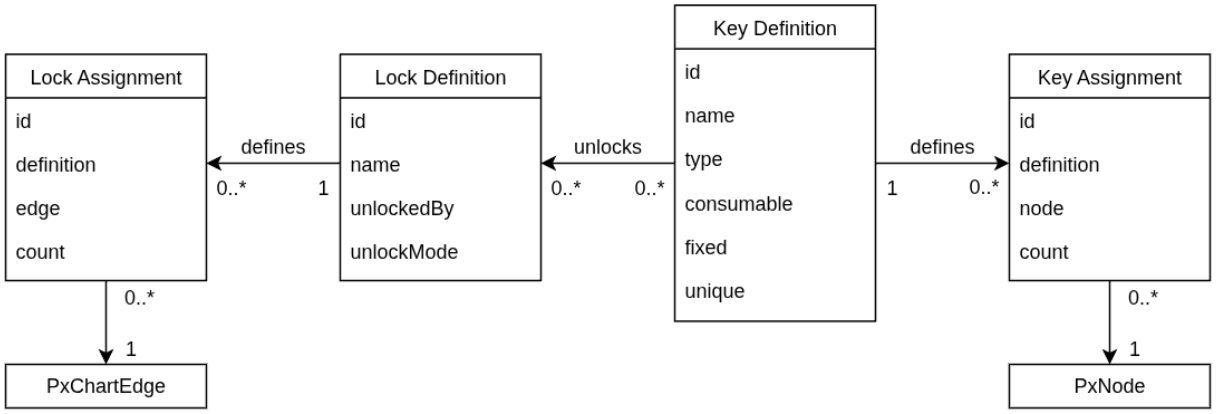

Figure 6: Data model for locks and keys

✏️ review multiplicities

- lock definitions and key definitions are used to represent different types of locks and keys, including the variations/aspects described in 2.4.2 (Taxonomy for Locks and Keys).
- in particular, a lock definition can be assigned multiple key definitions that unlock it. the same key definition can also be assigned to multiple lock definitions as an unlocking key type. additionally, both lock and key definitions can be reused across assignments.
- each pair of lock definitions and edge/key definition and node may have at most one assignment. assignments record `count` to model multiplicity.

4.3 Creation and Visualization of Locks and Keys

(Brown 2026): high-level visual representation. only focuses on locks and keys and where they are in relation to each other

to integrate into player experience chart: assign keys to nodes and locks to edges (to maintain clear structure)

4.3.1 Defining Types of Locks and Keys

- for both locks and keys: definitions separate from instantiations in nodes, analogous to PxComponents -> same logic behind both concepts
- both defined on same page so users can e.g. refer to details about key definitions while creating lock definitions and vice versa. page can be seen in Figure 7. layout: creation forms on the left (with help texts and indicators for necessary inputs), respective `ScrollAreas` for existing lock/key definitions next to it.

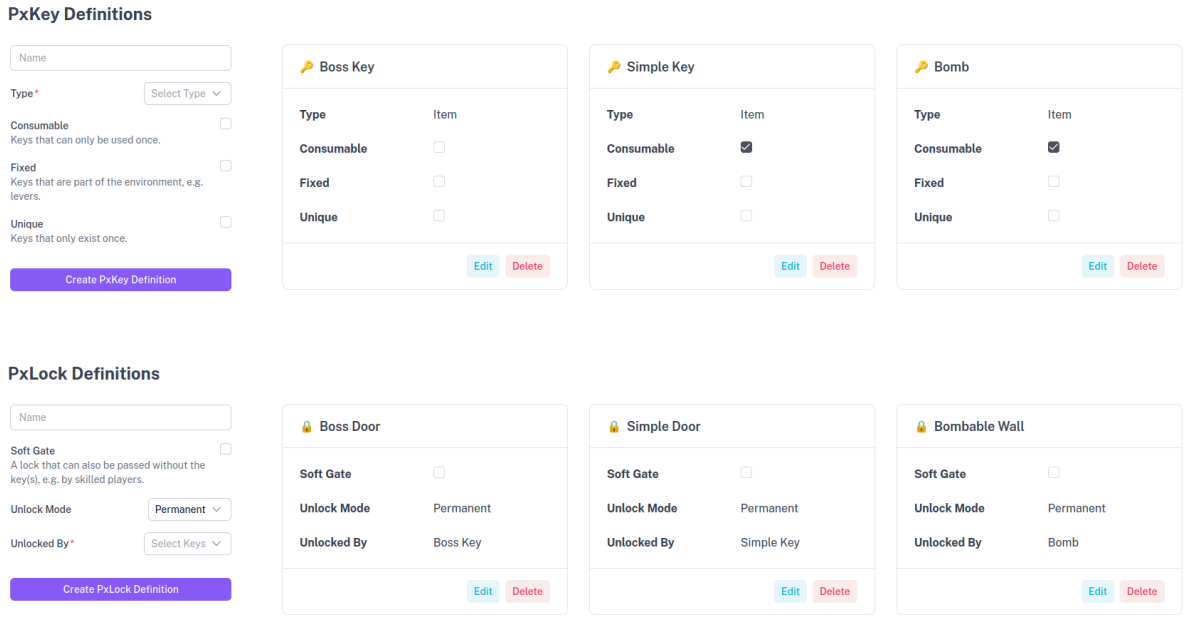

Figure 7: Page 'Lock and Key Definitions'

4.3.2 Modeling Keys

- multiple instances of same definition possible per node as defined in data model
- icon button in node (see Figure 9) opens modal (see Figure 8) where users can choose a key definition (from all definitions without an existing assignment) and respective count. both are technically required, count has default value 1.
- key assignments in nodes are represented by chips (count + definition name, see Figure 9), choice made due to compactness. chips have included button for deletion.

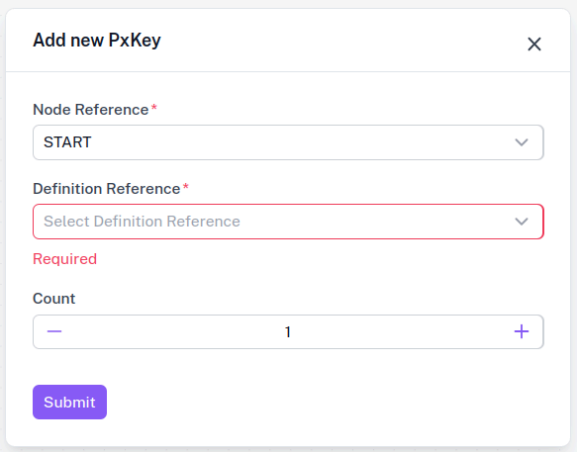

Figure 8: Modal for creating a key assignment for a node

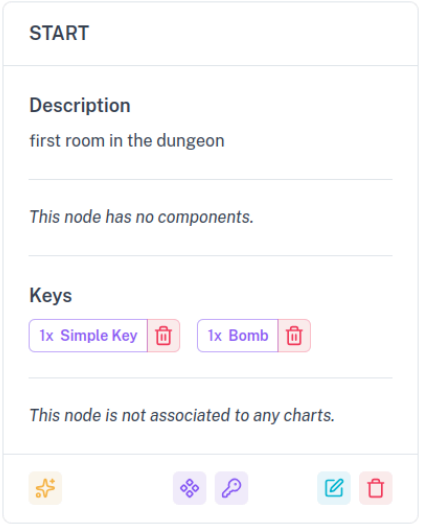

Figure 9: Visual representation of two key assignments in a node

- limitation: no editing of existing assignments. same as PxComponents, both would benefit from refactoring for easier editing.

4.3.3 Modeling Locks

- same as keys, multiple instances of same definition possible per edge, see data model.
- different from keys: edges are chart-specific and not represented in UI except for in the chart themselves. intention: make it clear that adding a lock to an edge is specific to that edge. vueflow framework does not offer built-in context menu for edges. possible future improvement: custom button implementation to simulate context menu (also for other actions like edge deletion). current workaround: buttons that are only visible if exactly one edge is selected. benefit: edge selection is easily detected.
- lock creation modal (see Figure 10): overview of all available lock definitions and instance counts on selected edge -> multiple and different kinds of locks can be edited/added in the same modal. key icon in each row shows unlocking key definitions.

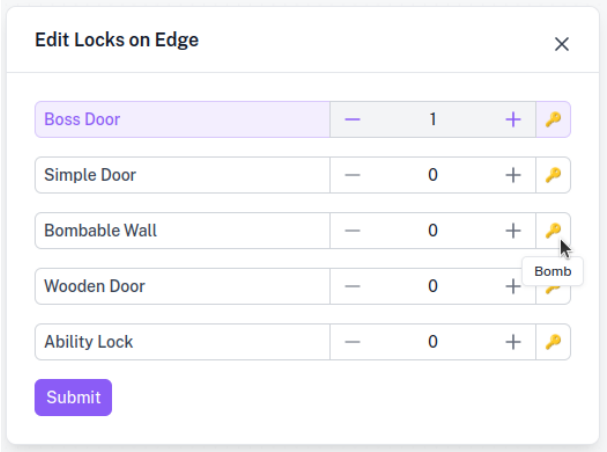

Figure 10: Modal for creating/editing lock assignments for an edge

4.4 Pathfinding with Locks and Keys

4.4.1 Basic Dijkstra with Locks and Keys

- goal: all locks assigned to edges in the path have to be unlocked by key(s) collected up to the edge in question
- inventory: track keys collected along path to determine whether encountered edges can be unlocked
  - inventory similar to (Aversa, Sardina, and Vassos 2015)
  - modeled via `PxKeySet` -> formally: mapping of key defs to counts
- basic form: inventory associated with a node reflects keys collected along shortest path to this node and keys made available in this node -> later extended (consumable keys, backtracking)

4.4.1.1 Unlocking Function

- `canUnlock()`: given (multi-)sets of locks and keys, determine whether locks can be unlocked
  - added complexity due to an edge potentially having multiple locks, and each locks potentially being unlocked by multiple keys
  - exponential blowup, but no optimization so far as average case is expected to be less complex ✏️ citation?

given set of keys in inventory K and locks on edge L:

```
canUnlock(K, L):
    if L = ∅
        return TRUE
    unlockingKeysets = required(l1) × ... × required(ln)
    if ∃ U ∈ unlockingKeysets where ∀ u ∈ U. u ∈ K:
        return TRUE
    else
        return FALSE
```

Pseudocode 1: Function checking whether a set of locks L can be unlocked by keys K

4.4.1.2 Example

✏️ add image (graph)?

Three locks assigned to one edge:

- lock A which can be unlocked by keys of type X or Y
- lock B which can be unlocked by keys of type X
- lock C which can be unlocked by keys of type Z

Leaving out consumability of keys: to unlock this edge, one could use the key sets $\{X, Z\}$ or $\{X, Y, Z\}$. If either is a subset of the available keys when encountering the edge, it can be unlocked.

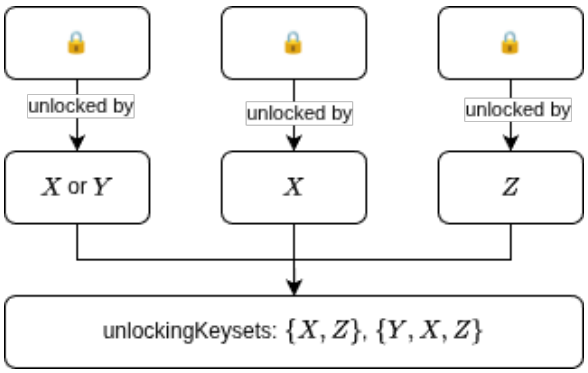

Figure 11: Calculation of unlocking keysets

- $I = \{X, Y\}$ -> false
- $I = X, Y, Z$ -> true

✏️ add example in pix:e

4.4.1.3 Full Algorithm

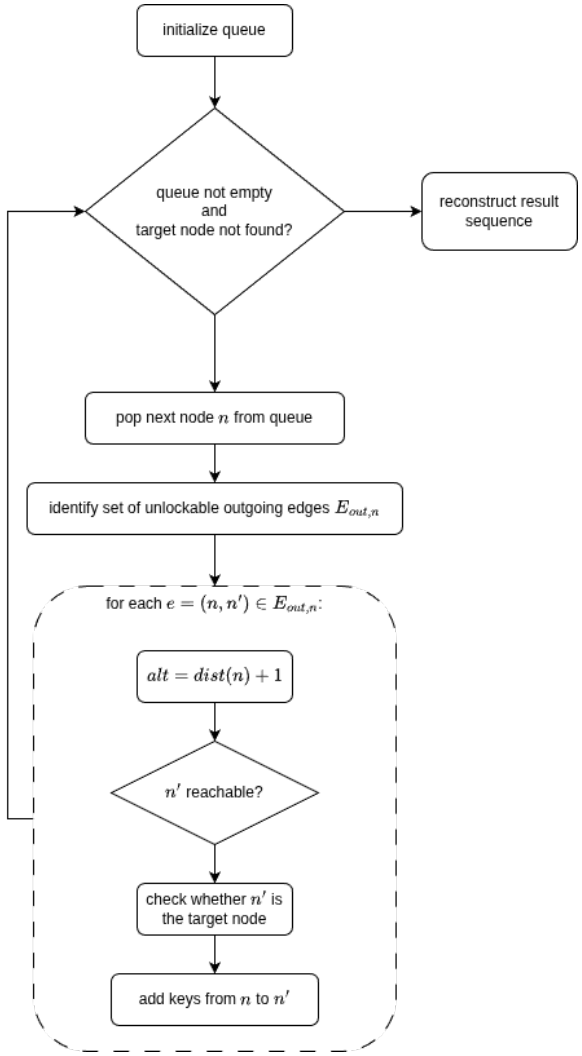

Figure 12: Main Loop of Pathfinding Algorithm with Locks & Keys (Basic)

4.4.1.4 Example Iteration

- ✏️ add small example of iteration

4.4.2 Fixed Keys

- goal: fixed keys can only be used within node they are assigned to
- limitation of current data model: no additional information for unlocking remote locks (would have to specify individual instances/assignments of fixed keys as unlocking a lock,m not just the type. example: two levers may open two different

doors). current implementation: can only unlock locks on outgoing edges of the node they are assigned to

implementation:

- do not add keys to inventory of neighbors
- ✏️ add example

4.4.3 Soft Gates

- goal: soft gates **can** be passed without the corresponding key, but this should be indicated in the path highlighting.

implementation:

- during pathfinding/unlock checks, ignore soft-gate locks
- after path has been found, re-trace it to determine whether keys are available or not
- ✏️ add example from pixe

4.4.4 Consumable Keys and Soft-Lock Detection

- goal: inventory should reflect usage of single-use keys
- variants of inventory -> idea of 'parallel universes'
  - nodes exist in different realities if they have different inventories
  - assign each node a set of possible keysets

implementation:

- maintain set of `PxKeySets` per node
- in `canUnlock`, consider multiplicity of consumable keys
- for updating inventory of neighbor nodes: remove consumed keys from inventory
  - if there are multiple possible constellations, add multiple keysets

4.4.4.1 Pathfinding with Consumable Keys

Extending Unlocking Check

given multiset of keys in inventory K and locks on edge L:

```
canUnlock(K, L):
    if L = ∅
        return TRUE
    unlockingKeysets = required(l1) × ... × required(ln)
    if ∃ U ∈ unlockingKeysets where U ⊆ K:
        return TRUE
    else
        return FALSE
```

Pseudocode 2: Function checking whether a set of locks L can be unlocked by keys K, considering consumable keys

- note: subset of multiset
- consider the same example as earlier, except now keys of type X are consumable, so to unlock the two locks A and B, now either two keys of type X or one key of type X and one of type Y are needed.

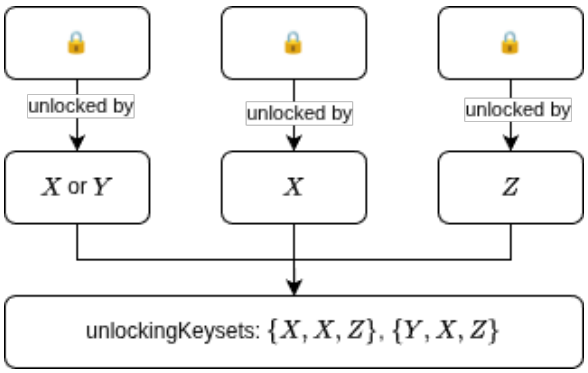

Figure 13: Calculation of unlocking keysets as multisets

- now: $I = \{X, Z\}$ -> false

Removing Consumable Keys from Inventory

given inventory I containing multisets of keys K and locks on edge L:

```
canUnlock(I, L):
    if L = ∅
        return TRUE
    consumableRequirements = locks with at least one consumable key unlocking
    unlockingKeysets = required(l1) × ... × required(ln)
    updatedInventory = {}
    for K_i ∈ I:
        if ∃ U ∈ unlockingKeysets with K_i ⊆ U:
            updatedInventory.push(K_i - { key∈U. consumable(key)})
    return updatedIinventory
```

Pseudocode 3: Function removing keys required to unlock all locks in L from K

✏️ review pseudocode correctness

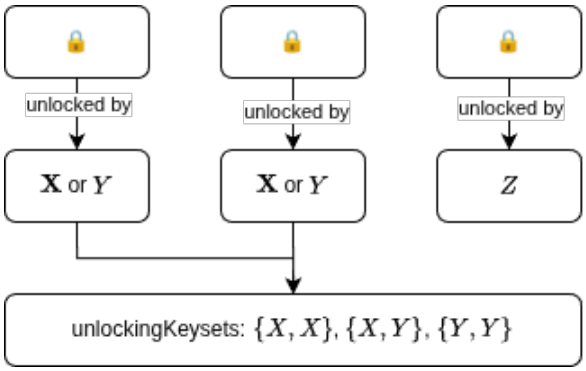

Figure 14: Calculation of unlocking keysets for key consumption

- $I = \{X, X, Z\}$ -> $\{Z\}$
- ✏️ example in pixe for consumable keys

4.4.4.2 Soft-Locks

definition: “it is possible to reach the goal state from every reachable state” (Mawhorter and Smith 2021):

- here: not nodes, but *states* in which nodes are visited

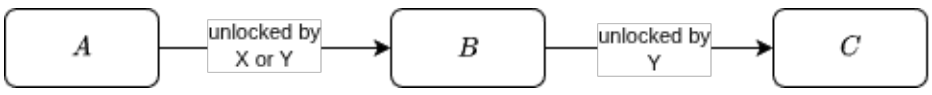

Figure 15: Example: Soft-lock

implementation:

- potential soft-lock state: node unlockable with some, but not all keysets
- when removing consumed keys: remove empty keysets entirely. then, compare inventory length before and after consumption
- ✏️ example in pixe for consumable keys

4.5 Bidirectional Edges and Backtracking

- goal: allow for longer/cyclic paths needed to collect keys
- another idea of parallel universes
- ✏️ add citation from julians paper?

implementation:

- alternative used here: track states as well as nodes. nodes are re-added to queue iff they have a different inventory assigned to them than in previous encounters -> only duplicate nodes as needed
  - for each node, maintain inventories it has been visited with
    - only re-visit with a different inventory
  - add limiting constant for revisits
  - also maintain unlocked edges

✏️ extend example

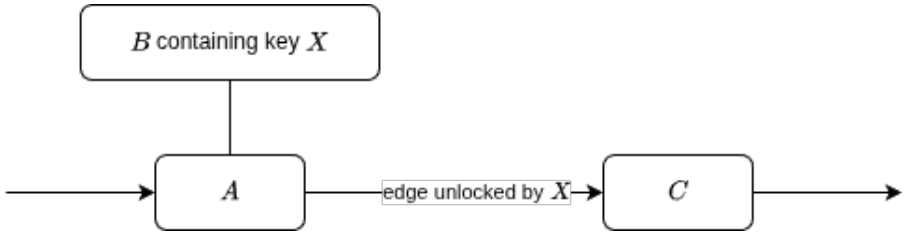

Figure 16: Example: Backtracking

- critical path: $A - B - A - C$

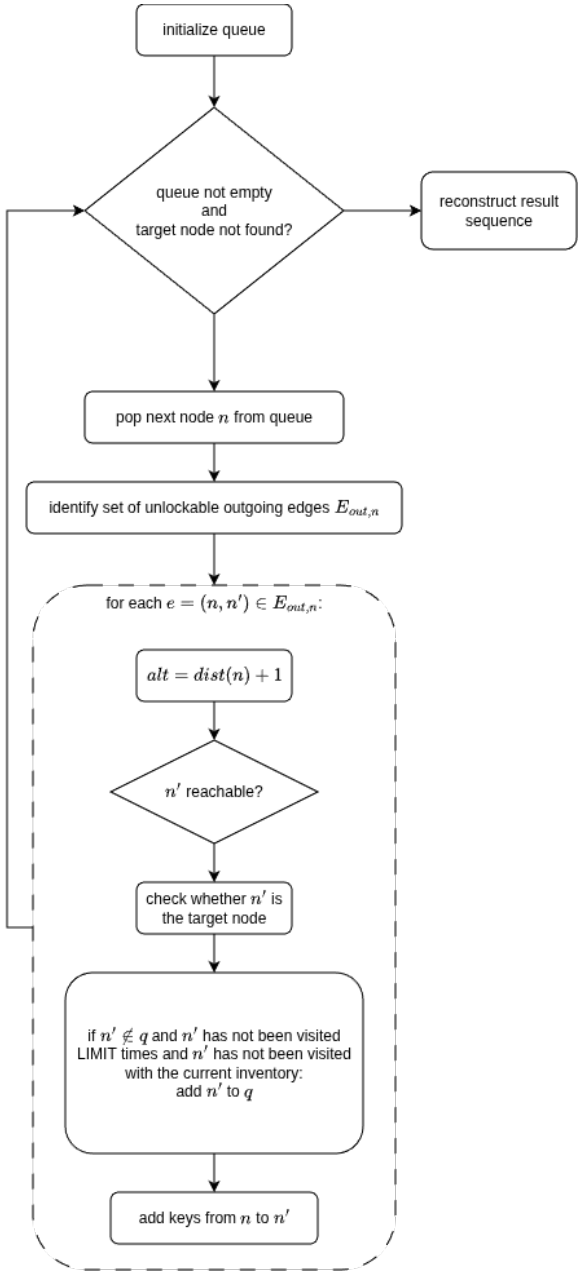

Figure 17: Main Loop of Pathfinding Algorithm with Locks & Keys (Extended)

4.5.1 Example

- ✏️ example screenshot from pixe

4.6 Options/Settings

- per-chart basis settings regarding pathfinding and locks/keys. for pathfinding, locks/keys can be enabled -> performance improvement for projects/charts without locks/

keys. for locks/keys: toggle for consumable keys being consumed during pathfinding (source for complexity in calculation), toggle for soft locks being visualized.

- extendability: tab structure (currently single tab only), so UI is set up for future extension with chart-specific settings (e.g., ✏️ think of examples)

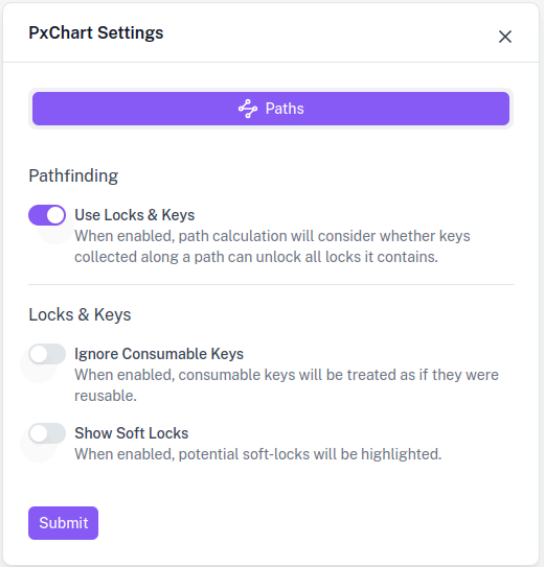

Figure 18: Settings modal

4.7 Code Architexture

- building on implementation for diagrams 3 (Path-Based Diagrams), pathfinding itself has higher complecity -> refactor into multiple files/composables

✏️ add diagram for code architecture + interactions?

modular architecture for extendability and maintainability. 4 main components:

- `PathCalculation`: main algorithm loop/logic
- `PathUnlocking`: logic specific to unlocking/key consumption
- `PathResult`: data type for results + computed values
- `PathStyling`: styling of edges/nodes based on results

4.8 Full Algorithm

- ✏️ make diagram of full algorithm?

4.9 Testing

- ✏️ describe test cases

4.10 Limitations and Future Work

- fixed keys for non-adjacent locks
- in pathfinding: temporary, reversible, collapsible locks
- difficulty estimation ✏️ add citation
- recommendations for lock/key types (Dormans 2017)
- LLM-based consistency check with descriptions
- implementation: currently done in frontend. may benefit from porting to backend + specialized algorithm. trade-off: maintainability/extendability

5 Evaluation

- user study. this section described design/setup and results.

5.1 Methodology

5.1.1 High-Level User Study Design

- joint user study/team effort. first part of study was concerned with usability, participants asked to model zelda dungeon in tool (including locks & keys) -> participants were already familiarized with context and basic functions of the tool when approaching the tasks described here. task-specific functionalities (pathfinding + solvability-related highlightings, diagrams) were shown to participants prior to respective tasks.
- task-based within-subject: StatePx vs Miro
  - two task variants, varying order of tools + dungeons -> 4 groups:

	Miro first	**StatePx first**
Variant A first	MA	PA
Variant B first	MB	PB

Table 1: Definition of the 4 user study participant groups

- use case: zelda dungeons (Summerville et al. 2016) -> ensure realistic tasks. data pool big enough to choose 2 comparable dungeons per task. same game: consistency in terms of details like item/key types, dungeon structure, etc. dataset provides graphs for each level and full layout/map of dungeons.
- target demographic: people with some understanding of game design -> recruitment: university lecture in games engineering study course, also computer science students who are familiar with videogames
- complete tasks (all dungeons) + answer keys in appendix ✏️ add to appendix

5.1.2 Modeling Zelda Dungeons

5.1.2.1 General Modeling

- rooms numbered based on location in map (approx from bottom left to top right)
- some rooms spanned multiple screens and were split (e.g. 9-1, 9-2, 9-3) to more accurately reflect pacing (e.g. enemies that are technically in the same room but do

not spawn/become visible and defeatable until the respective screen/part of room is entered)

✏️ image showing room numbering

5.1.2.2 pix:e vs Miro

pix:e: rooms: node. use in-built functionalities for keys/items and other room data (components). visualization: as described previously

Miro: rooms: sticky notes. write all information into sticky notes. heavier use of emoji encoding: to more easily differentiate between different key types/lock types

comparison: pix:e has more dedicated functionality + logic, but ui is less mature (zoomed out view, editing features). emoji encoding in Miro.

5.1.3 Solvability Tasks

- larger Zelda dungeons
- Link's Awakening LA_3, LA_4 (Summerville et al. 2016)
- selection criteria: size -> non-trivial while still being feasible manually in Miro. comparable size (35 vs 41 nodes) + complexity (outgoing ranks of nodes), similar number of keys/locks. ✏️ add details

✏️ images: original maps/graphs, versions in Miro/pix:e

- slight adaptations: ✏️ add adaptations
  - structural: ✏️ add structural adaptations
  - information: remove puzzles, "vision" edges, enemies
- added issues: 6 for each dungeon, of varying difficulty
  - 2x missing key(s), 1x no outgoing edge, 1x unreachable due to edge directionality, 2x softlock
- task: find as many issues -> specified kinds of issues
- participants were allowed to use tool functionalities
  - Miro: e.g. draw in paths, use pens/sticky notes to mark found issues
  - pix:e: pathfinding functionality w/ highlighting, fixing issues after reporting

5.1.4 Pacing Analysis Tasks

- smaller Zelda dungeons
- Link's Awakening LA_1, LA_2 (Summerville et al. 2016)
- selection criteria: size, number of keys/locks. comparable size + complexity ✏️ add details .

- additional criterium: little backtracking, clear division into three sections that can be used for comparative analysis of component values along (sub-)paths:
  - A: start to ability
  - B: ability to boss key
  - C: boss key to boss fight

✏️ images: original maps/graphs, versions in Miro/pix:e

- adaptations: ✏️ add adaptations
  - structural: ✏️ add structural adaptations
  - information: remove puzzles, “vision” edges. locks/keys stay.
  - additional information: completion time and enemy count based on YouTube walkthrough (100% completion to get data for all rooms) ✏️ add ref to walkthroughs .
- task: answer questions about pacing. includes comparisons of path sections, overall enemy distribution in nodes:

1. Is the number of enemies steadily increasing throughout the dungeon?
2. Does it take longer on average to complete rooms in Section A than in Section B?
3. Are there more enemies in Section C than in Section B?
4. Which room most likely contains a miniboss?
5. Between which two consecutive rooms does the number increase the most?
6. Which room after a high-intensity encounter has notably fewer enemies or lower completion time?

- participants were allowed to use tool functionalities
  - Miro: e.g. draw in paths, use pens/sticky notes to note down calculations and answers
  - pix:e: pathfinding functionality w/ highlighting, diagrams

5.1.5 Collected Data and Processing

- demographics: age, gender, background, occupation experience, previous experience with whiteboard tools. collected via Google Forms

✏️ add complete demographics

- recorded during tasks: issues found/questions answered + timestamps, comments, clarifications/hints, observations about tool use
  - semi-structured live protocol, written in Markdown with YAML frontmatter for participant data (id, group)
- separate pix:e accounts/miro boards for participants -> edits

- additionally: UEQ-S (Schrepp, Hinderks, and Thomaschewski 2017) (short version recommended for evaluations of multiple versions of a system), questionnaire comparing Miro and StatePx. collected via Google Forms

✏️ disclaimer: questionnaires team effort

post-processing:

- manual tagging of task meta data per participant and task: total counts/points based on protocol. parsing script to transform into csv
- codification of google form free-form answers (whiteboard experience, education, occupation)

-> different sets of data

5.2 User Study Results

5.2.1 Participants

- total number: 24
- demographics:
  - ages 19-32, average 23.625
  - ✏️ educational/occupational background
  - 15 indicated previous experience with Whiteboard tools (62.5%)
- group breakdown?

5.2.2 Results Solvability

✏️ compare A/B groups and orderings

5.2.2.1 Data Processing

- spoken answers recorded in protocol -> manual count of total identified issues, manual check against answer key and count of true positives
- inspection of miro boards -> modifications to boards and if so, which ones

✏️ add disclaimer human error

5.2.2.2 Task Completion

- total number of identified issues: overall less issues claimed in pix:e, both across all participants as well as within participants, see Figure 19. difference is statistically significant (according to Wilcoxon signed-rank test, $T = 48.5$, $p = 0.00564929988211376$, ✏️ effect size?)

- correctly identified issues (f1): significantly lower in pix:e overall, again both across and within participants (according to Wilcoxon signed-rank test, $T = 34.0$, $p = 0.0009066055498154164$ ✏️ effect size?), see Figure 20.

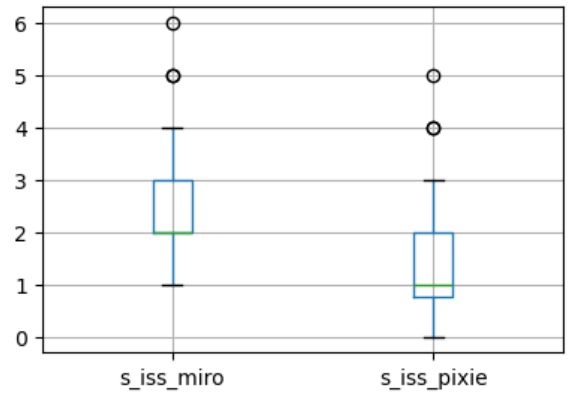

Figure 19: Distribution of total number of identified issues by tool

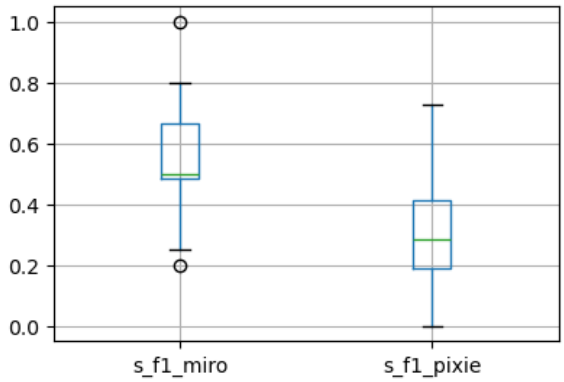

Figure 20: Distribution of F1 scores by tool

- in pixie: more issues claimed by people with whiteboard experience, see Figure 21. median 1 for participants without whiteboard experience, 2 for participants with whiteboard experience, Mann-Whitney $U = 96.5$, $p < 0.05$. ✏️ effect size . however, participants with whiteboard experience still identified significantly fewer issues in pixie than in Miro (Wilcoxon signed-rank test, $T = 97.5$, $p < 0.05$)
- significantly higher pix:e F1 in people with whiteboard experience, see Figure 22. F1 medians 0.25 and 0.285, Mann-Whitney $U = 99.5$, $p < 0.05$ ✏️ effect size?

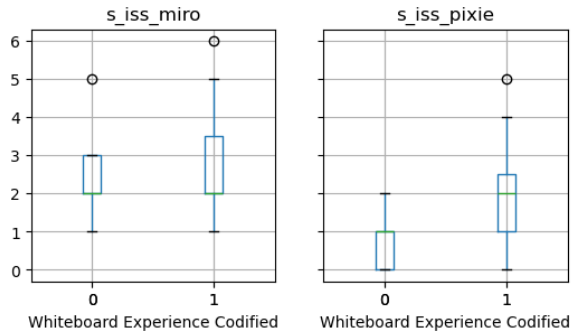

Figure 21: Distribution of total number of identified issues by tool and indicated whiteboard tool experience

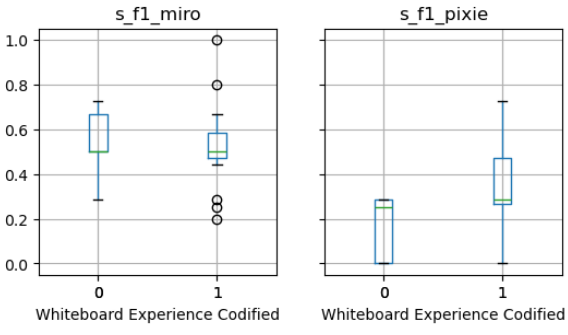

Figure 22: Distribution of F1 scores by tool and indicated whiteboard tool experience

- control for actual usage of Miro functionality (6 participants): 3x marking issues, 1x drawing reachability, 2x inventory tracking

- no obvious difference in task completion in Miro depending on tool use, see Figure 23 and Figure 24. total issues identified: median 2 without Miro usage, 2.5 with Miro usage, Mann-Whitney $U = 57.5$, $p = 0.4169872469244896$ ✏️ effect size? . F1: median 0.5 with Miro usage, 0.39285714285714285 without Miro usage, Mann-Whitney $U = 57.5$, $p = 0.4169872469244896$ ✏️ effect size?

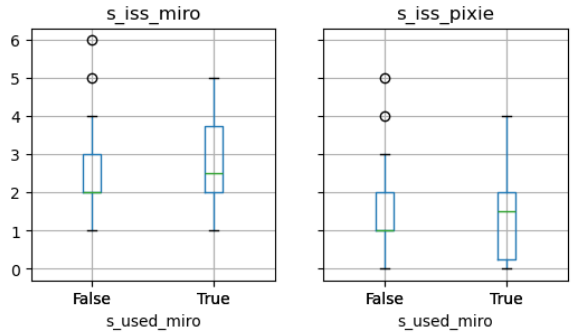

Figure 23: Distribution of total number of identified issues by tool and use of Miro functionalities

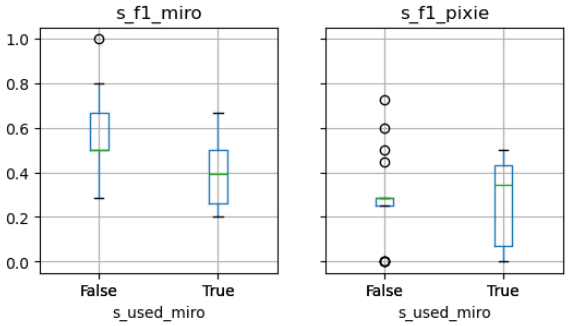

Figure 24: Distribution of F1 scores by tool and use of Miro functionalities

- overall lower task completion and answer quality in pix:e than Miro. within pix:e, participants with whiteboard experience achieved better results than participants without. within Miro, participants using Miro functionalities did not achieve better results than participants not using Miro functionalities.
- possible explanation/interpretation: similar to first task. pix:e caters better to power users. Miro functionalities (without involved custom setup/templating) seem to be less suited to support users with this specific analytical task type.
- ✏️ reference feedback

5.2.2.3 Recorded Times

- ✏️ times to first identified issues

5.2.3 Results Pacing

✏️ add disclaimer human error ✏️ effect sizes ✏️ compare A/B groups and orderings

5.2.3.1 Data Processing

analogous to solvability task:

- spoken answers recorded in protocol -> manual count of total answered questions, manual check against answer key and count of correct answers.
- inspection of miro boards -> modifications to boards and if so, which ones

5.2.3.2 Task Completion

- total number of answered questions: no indication of significant difference in distributions according to Wilcoxon signed-rank test ($T = 96.5$, $p = 0.7473513186644183$).
- correctly answered questions: also no indication of significant difference in distributions. Wilcoxon signed-rank test: $T = 98.5$, $p = 0.8058983773663436$
- slight trend: median higher in pix:e, but distribution skews higher for miro.

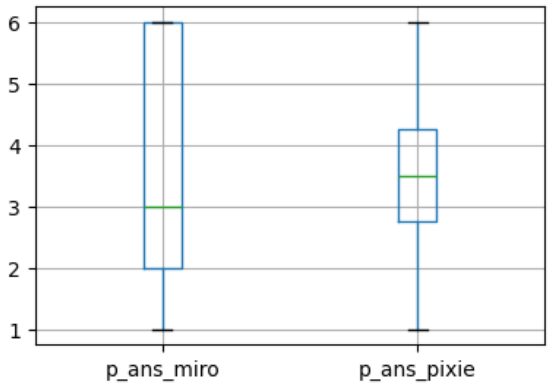

Figure 25: Distribution of total number of answered questions by tool

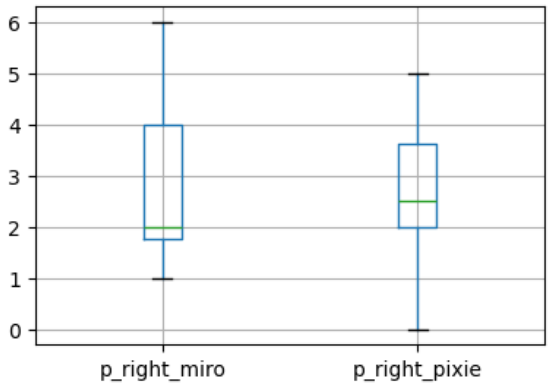

Figure 26: Distribution of number of correctly answered questions by tool

- within pix:e, participants with whiteboard experience tended to answer significantly more questions (see Figure 27): median 4 with whiteboard experience, 3 without whiteboard experience, Mann-Whitney $U = 103.5$, $p = 0.015198521622538362$). they also gave significantly more correct answers (see Figure 28): median 3 with whiteboard experience, 2 without, Mann-Whitney $U = 103.0$, $p = 0.01660439188879628$)
- slight skews in median/distribution: number of answered questions higher in pix:e than miro for participants with whiteboard experience, total number and number of correct answers lower in pix:e than Miro for people without whiteboard experience. however, differences in distribution do not seem to be statistically significant (Wilcoxon signed-rank test: $38.0, 0.325888948764388$ for total answers in pix:e vs Miro from participants with Whiteboard experience, $(15.0, 0.20703125)$ for total answers from participants without whiteboard experience)

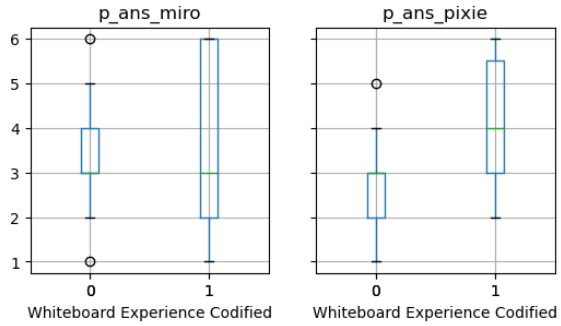

Figure 27: Distribution of total number of answered questions by tool and indicated whiteboard tool experience

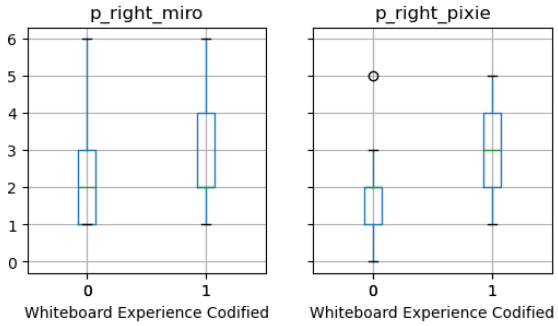

Figure 28: Distribution of number of correctly answered questions by tool and indicated whiteboard tool experience

- control for usage of Miro functionalities (6 participants): 1x sticky note color change to highlight a node/room, 4x calculations in textbox/sticky note/via pen, 1x recording path in textbox
- median of total answered questions and correctly answered questions in Miro lower for participants using functionality in Miro (see Figure 29 and Figure 30), but not significantly so (Mann-Whitney test: $U = 32.5$, $p = 0.07540184146884174$ for total answers, $U = 33.5$, $p = 0.08497339144083382$ for correct answers).
- also, median for total number of answered questions and number of correctly answered questions larger in pix:e than miro for participants using Miro functionality, but not statistically significant (Wilcoxon signed-rank test: $T = 11.0$, $p = 0.1875$ for total answers, $T = 13.5$, $p = 0.296875$ for correct answers)

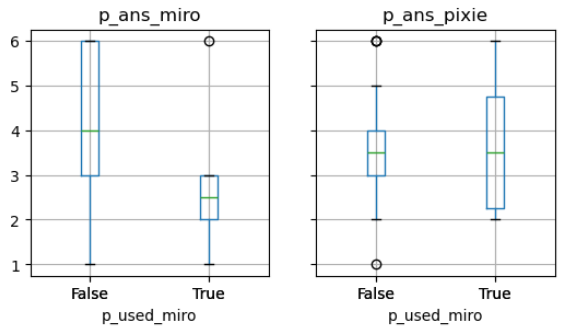

Figure 29: Distribution of total number of answered questions by tool and use of Miro functionalities

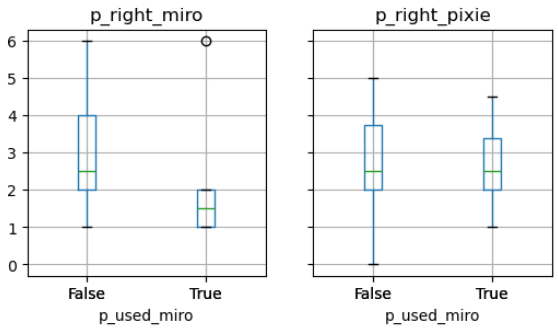

Figure 30: Distribution of number of correctly answered questions by tool and use of Miro functionalities

- overall, no significant difference in task completion and answer quality between tools. within pix:e, participants with whiteboard experience achieved better results than participants without. within Miro, participants using Miro functionalities did not achieve better results than participants not using Miro functionalities.
- possible explanation/interpretation: similar to first task. pix:e caters better to power users. Miro functionalities (without involved custom setup/templating) seem to be less suited to support users with this specific analytical task type.

5.2.3.3 Recorded Times

- ✏️ time per question

5.2.4 Results UEQ-S

- using provided analysis tool based on benchmark for the UEQ-S (Hinderks, Schrepp, and Thomaschewski 2018) ✏️ what is the benchmark?
- 8 items divided into 2 categories: pragmatic quality ("relate to the tasks or goals the user aims to reach") and hedonic quality ("related to pleasure or fun while using the product"). overall scale ranges from −3 to 3.

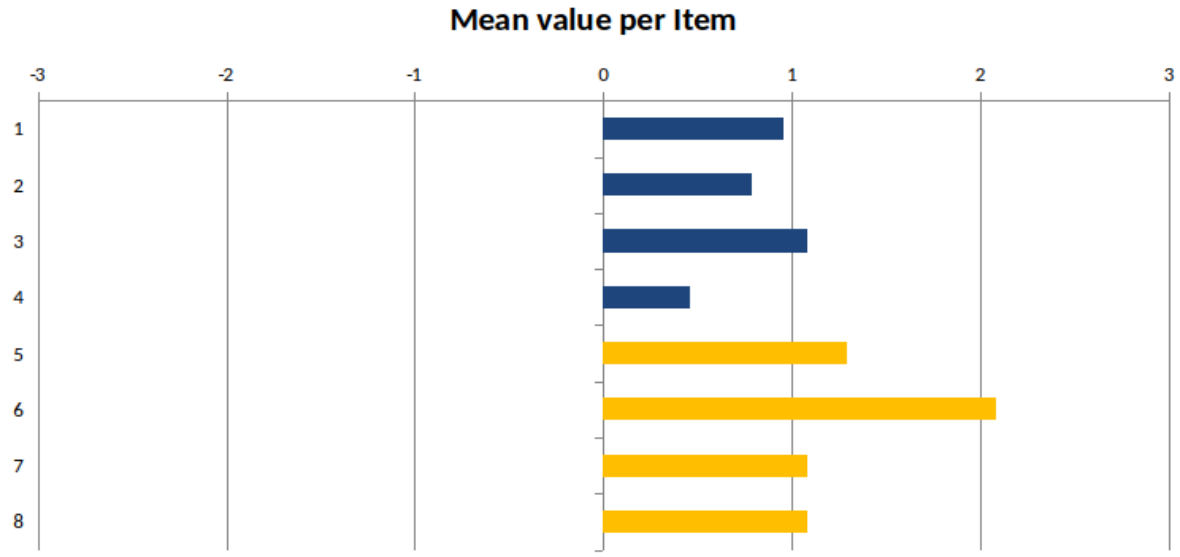

Figure 31: Mean values per item in the UEQ-S

- weakest item: confusing vs clear
  - potentially relates to: UI quirks, lock visualization
- pragmatic quality: mean 0.823 (-> below average, i.e. better than 25% of benchmark), hedonic quality: mean 1.385 (-> good, i.e. better than 75% of benchmark), overall: mean 1.104 (-> above average, i.e. better than 50% of benchmark)

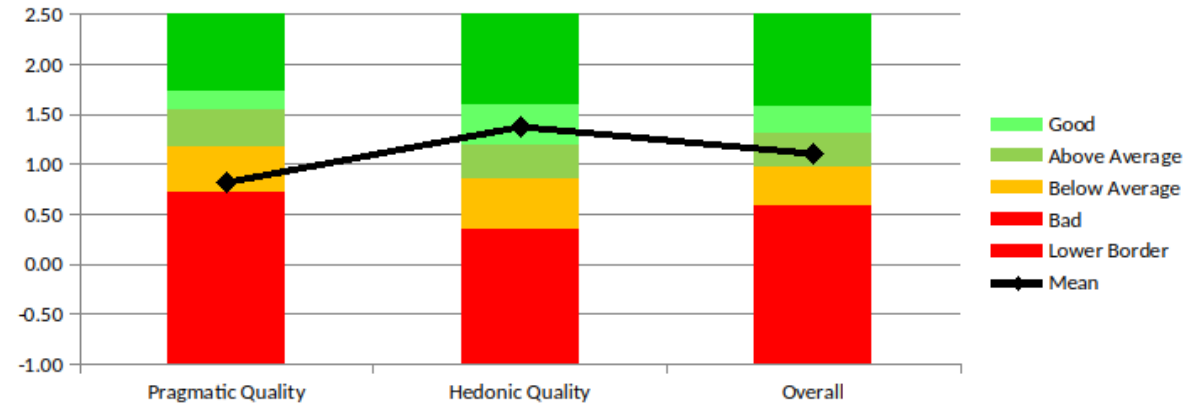

Figure 32: Comparison of UEQ-S scores against benchmark

✏️ control for whiteboard experience?

5.2.5 Results Comparative Questions

- C-Q1: "Compared to Miro, the system made it easier to analyze dungeon designs. "
- C-Q2: "Compared to Miro, I felt more confident in my answers. "
- C-Q3: "Compared to Miro, I was able to complete the tasks more efficiently. "
- C-Q4: "If I were analyzing game levels in practice, I would prefer using the system over a general-purpose whiteboard."

5-point Likert scale, fully disagree (1) to fully agree (5)

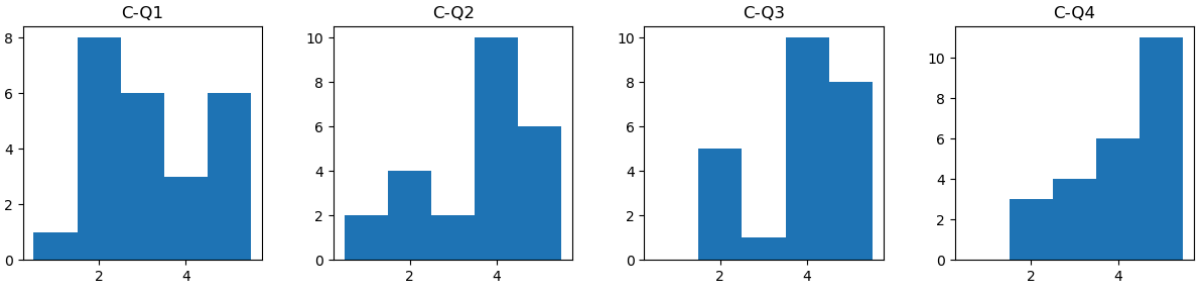

Figure 33: Answer distributions for system preference questions, across all participants

- overall, participants indicated preference for statepx over miro, mixed answers with a slight preference for pixie in first three questions
- in line with high hedonic quality reported in UEQ-S
- higher score variance/split observed in participants with whiteboard tool experience in the first three questions, see B.1 (Results of Comparative Questions by Whiteboard Experience). for last question, participants with previous whiteboard experience in particular indicated strong preference for pix:e ✏️ back this up . in line

with high hedonic quality. might be because “power users” are better equipped to recognize advantage of tool compared to whiteboard tools.

5.2.6 Written Feedback from Participants

- context: once after first part of user study (not described here ✏️ add to appendix + reference?), once at very end of study
- see complete (relevant) freeform answers in Appendix B.2 (Written Feedback)

5.2.6.1 Usability/UI

- issues/confusion when adding locks (5x) due to menu placement
- addition of keys not included in right-click menu of nodes (2x)
- low contrast between background and nodes (6x)

5.2.6.2 Functionality

- locks not visually distinguishable (6x)
- confusion about diagram tick labels
- sum/average in diagrams, more interactivity in general
- unclear highlighting in pathfinding (2x)

5.2.7 Observed Behavior and Verbal Feedback from Participants

- all protocol data, thus may contain human error
- contains: verbal comments/questions while completing the tasks, specific approaches to tool use, verbal feedback
- 5 participants required clarification that all locks were visualized by the same symbol (4 of them completed the task in Miro first, thus may have been primed to expect similar visualization)
- 10 participants did some form of successive pathfinding in pix:e
- 3 participants (all Miro-first groups) asked whether enemies respawn for the pacing task. while unclear in task description, still notable that only Miro-first participants asked this. setup in pix:e may have primed participants to understand the respawn implicitly and apply the same principle in Miro. -> potential improvement for diagrams setup, users should be prevented from making/falling victim to implicit assumptions!

5.2.8 Validity of A/B Test

- compare results across groups -> statistical test? (ANOVA for 3+ groups)
- compare demographics across groups

5.3 Limitations

- data collection: human errors
- not all functionality included (e.g. soft gates, creation of component/lock/key definitions, broder functionalities of pix:e)

5.4 Post-Study Improvements

- based on feedback/observations

5.4.1 Improved Visual Representation of Locks & Keys

- initial version: all locks on edges represented by same symbol respectively. high number of participants feedbacked that distinct visualization of different lock types would improve the experience, or expressed/showed confusion due to the initial representation.
- addition to data model: `symbol` for both locks and keys, can be selected in picker when creating definitions. defaults: 🔒 and 🔑.
  - key assignments show symbol in preview, tooltip shows name of definition on hover
  - edges show symbols of all assigned locks as label. tooltip more involved in current framework. instead: legend popover
- legend: bottom right corner of chart. can be shown/hidden via button. shows overview of available lock + key definitions in separate tabs, including most important informations for users (e.g. symbol, name, for keys: consumability, for locks: unlocking key definitions)

✏️ add screenshots

5.4.2 Reachability Analysis

- many participants (number) either explicitly reported missing a reachability analysis functionality in open feedback or were observed to conduct a reachability analysis during the solvability task by selecting various end nodes for the given start node and subsequently identifying locked edges as those between a node reachable from the start and a node not reachable from the start (10 participants). ✏️ add numbers
- dijkstra algorithm tracks distances of nodes from start nodes. initial value: `Infinity`, i.e., all nodes with distance < `Infinity` are reachable. -> simply filter after pathfinding has concluded
- in case of failed pathfinding: highlight unreachable nodes ✏️ confirm/wait for julians pr feedback

✏️ add screenshots

5.4.3 Increased Contrast of Nodes Against Background

- high number of participants reported low contrast between nodes and background as a usability issue, especially when zoomed out ✏️ add numbers
- new color in css theme for background for both dark and light mode

✏️ add screenshots

6 Discussion

6.1 Implementation

- lots of groundwork with focus on extendability and flexible design -> hopefully nice to build upon
- many improvements and new features already identified during implementation
- limited by time constraints and framework unfortunately

6.2 Evaluation

- summary of results: no clear advantage in task completion in pix:e, though it did produce better results for participants who indicated previous experience with whiteboard tools for both tasks. high hedonic quality in UEQ-S and preference for pix:e over Miro for the presented tasks. freeform feedback provided many meaningful pointers for possible improvements, some of which were already implemented, and generally expressed excitement about the potential of the tool.
- possible explanation/interpretation: tool better suited for power users (extrapolated: people familiar with tool-supported game design). in line with feedback regarding the tool having a learning curve -> may be easier to use when building on pre-existing knowledge. but also, when thinking about target users, this may be an additional incentive for Miro users to switch.

7 Related Works

8 Future Work

8.1 Implementation

✏️ collect from other chapters

8.1.1 UI/UX

- easier editing of component/key assignments ✏️ add reference
-

8.1.2 Existing Functionalities

8.1.3 Extended and Additional Functionalities

branching out:

- using LLMS:
  - LLM-based pacing analyses
  - LLM-based creation of locks/keys from description
- path-finding, path-based analysis:
  - structural checks
  - maybe something with petri nets? ✏️ research petri nets
  - saving/comparing paths
- solvability:
  - path-agnostic analysis (single button “analyze” to check for any solvability issues)
- diagrams:
  - ✏️ think of something

8.2 Other

- larger case study where system is used more in-depth/over a longer amount of time ✏️ find citation where someone did this

9 Conclusion

A User Study Tasks and Descriptions

B User Study Data

B.1 Results of Comparative Questions by Whiteboard Experience

The following plots visualize the answer distributions to the four questions comparing Miro to StatePx stratified by indicated whiteboard experience on a shared y-axis.

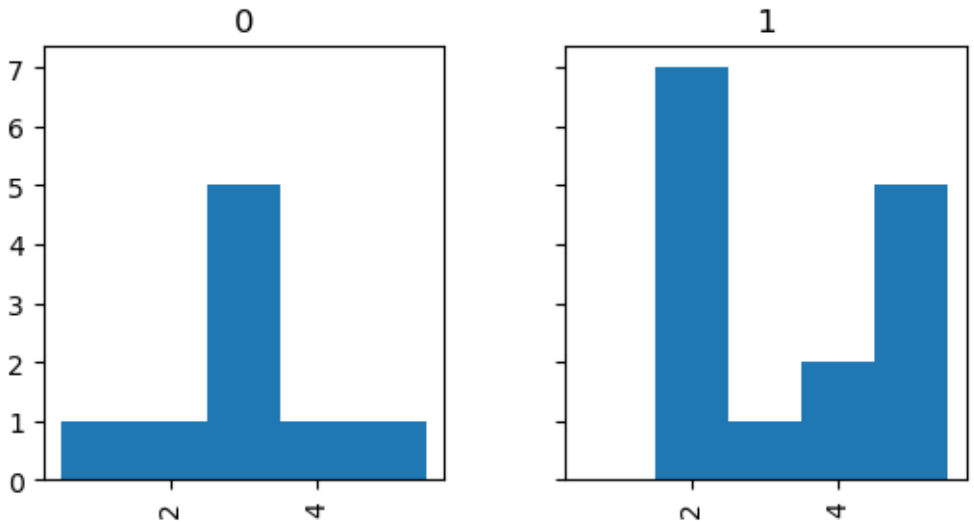

Figure 34: Distribution of answers to C-Q1 by indicated whiteboard tool experience

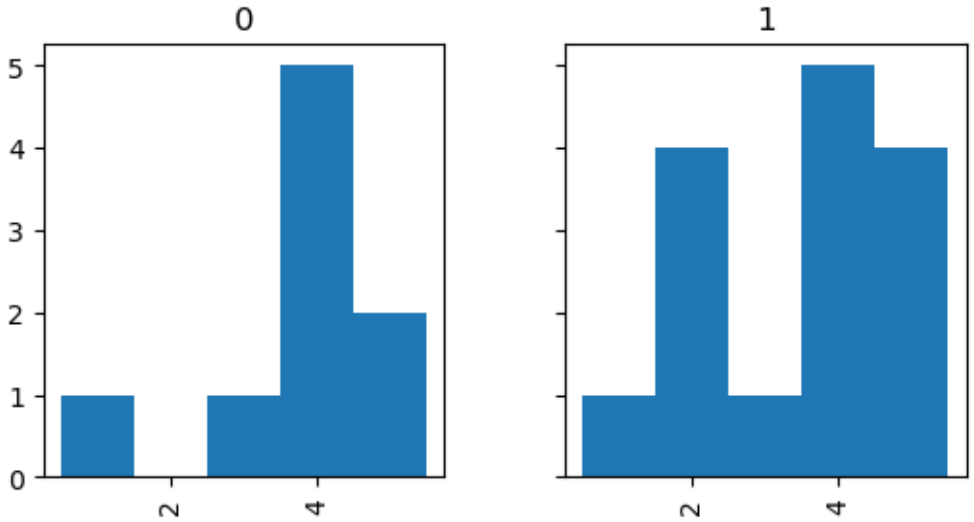

Figure 35: Distribution of answers to C-Q2 by indicated whiteboard tool experience

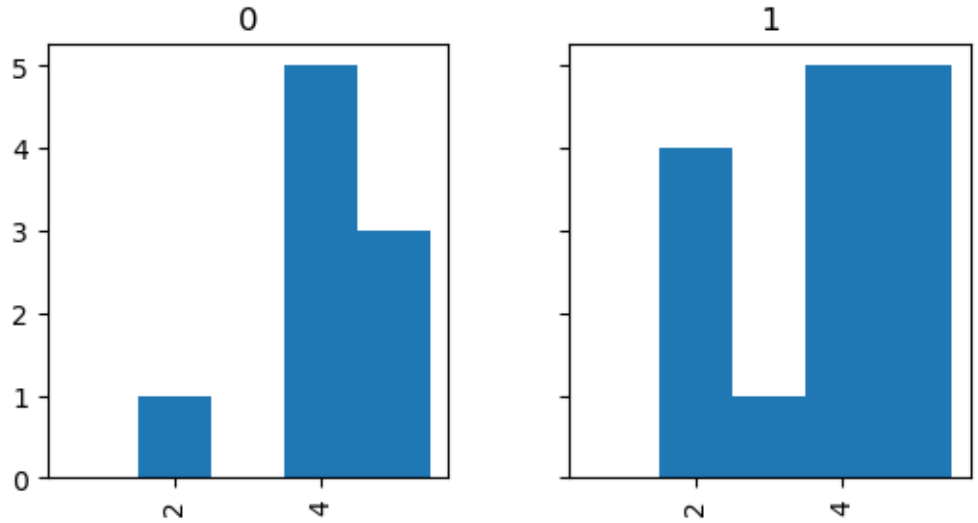

Figure 36: Distribution of answers to C-Q3 by indicated whiteboard tool experience

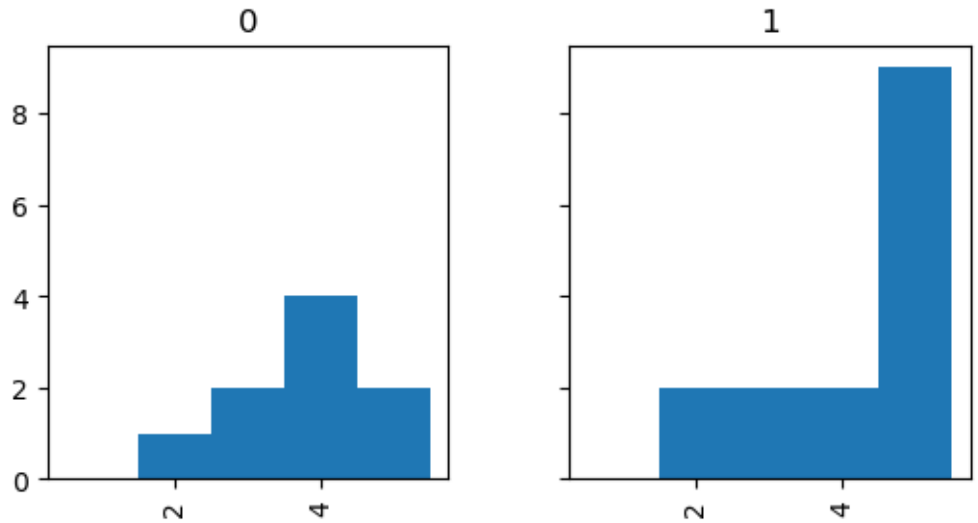

Figure 37: Distribution of answers to C-Q4 by indicated whiteboard tool experience

B.2 Written Feedback

B.2.1 After Part 1

After first part of user study: "Was there anything about the modeling interface that you found confusing or frustrating?" filtered here for comments relevant to work in this thesis (mainly locks & keys)

- I often had to double-click on an object, like a button or an edge, in order to activate it or change its properties.
- I didn't see where I could add locks

- I was to dumb to figure out how to add a key, also the right click context actions could be more consistent
- No multi-select via drop-clicking was frustrating. Also didn't notice lock option on edges until the end. "Key" button for bombs too.
- Adding locks to edges is triggered with a separate menu, which did not pop up by clicking at the edge, having a less natural flow as for the rooms, where edit options where shown in the room box directly.
- The button for adding keys is not in the right-click menu.
- There could be more information e.g. in tooltips
- Sometimes, when adding a new component, the system was showing weird values in their place, taking roughly a second to update. Also, when clicking on an edge, I wanted to know what kind of lock it showed.
- Locks were a bit weird to set up, the two buttons in the top left felt not quite in place. Also some features have right click context menu actions while others don't.

B.2.2 After Part 2

After entire user study: "Anything else you want to tell us?"

- The locks need to be differentiable by look
- It's nice to use but it takes some time to get used to imo
- Would have liked to have a "total time needed to reach room n" when selecting a path. Also it was slightly confusing that the diagramme didn't show the rooms on the path in order on its x axis.
- Tool did not make it clear which locks had which type (UI issue). I felt confident answering questions for the "direct" path, but manual intervention would be required to analyze indirect paths. Thus I did not feel confident answering questions in full generality.
- Please for gods sake put more contrast between the nodes/the graphical outline of the nodes and the background it is very difficult to exactly see the nodes which lead to some confusion on my part
- I feel like so much potential is wasted not in the design of the system but the missing polish, div borders disappearing when zoomed out too far, no different visual indicators for walls, locks etc. Also a sum and average funtction would be huge for the pacing analysis
- i wasn't really sure what happened in the path finding when i tried to get from start to finish and no node had a rim and random paths lit up, it does need some time

to get used to but i think it could really help upon building dungeons; a few more statistics would be fun to see averages

- I would have liked to be able to interact more with the diagram, e.g. some on specific parts of the x achsis.
- Probably the biggest issue that I had was the readability of it. On Miro the rooms were on a yellow note, having contrast with the background, with only essential information simpled down. On the new system, the rooms are too big and apart from each other, also blending with the background, and the information took more time to read and comprehend
- it's easy to accidentally moving nodes; zooming out looses details such as the borders around the boxes that are the nodes in the graph, instead of being more coarse-grained.
- When getting a path from one node to another, I would like to get more information about the intermediate states the system had on the nodes in between.
- in miro i liked that i could see the number of enemies or the types of doors etc. directly on the node even when zoomed pretty far out. the different components had different icons and colors (because they were emojis) which made it easy to distinguish them. Also the name of the nodes was bigger and the edges were more bold so i could read everything clearly even when zoomed out pretty far. in pix:e i could barely read anything except if i was zooming in a lot which hindered the development of a mental map. Though, the pathfinding tools and diagrams for enemy number and time in pix:e were pretty useful and in miro i was requiring more mental effort to keep track of the enemies and times etc.
- It was way easier to tell the type of a lock in Miro because of the different symbols
- The nodes are hard to distinguish from the background when zooming out because of a low contrast and thin outlines. Otherwise I would've found relevant nodes much faster.
- In its current state the system lacks clear indicators of what exactly is being highlighted/which paths are analyzed ect. - stronger graphical indicators would completely change the experience for me
- More functionality would be nice: Better visual cues (as in the miro board) for different types of doors, etc., cumulative graphs for the function, better readability of the graph (it was not so clear to see graph node A in the function graph anymore when selecting path from A to B to C in task 2)

- The tool definately has great potential, but there are some minor nitpicks which for me made it less usable as miro. If those are fixed im confident it can and will be better

C Further Evaluation

List of Figures

List of Tables

Bibliography

Calvin Ashmore and Michael Nitsche. 2007. The Quest in a Generated World. In *Proceedings of DiGRA 2007 Conference: Situated Play*.

Davide Aversa, Sebastian Sardina, and Stavros Vassos. 2015. Path Planning with Inventory-Driven Jump-Point-Search. *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, 11(1):2–8.

Mark Brown. 2026. How My Boss Key Dungeon Graphs Work | Patreon. https://web.archive.org/web/20260226034644/https://www.patreon.com/posts/how-my-boss-key-13801754.

Joris Dormans. 2017. Cyclic Generation.

Julian Geheeb, Daniel Dyrda, and Sebastian Geheeb. 2024. PaceMaker: A Practical Tool for Pacing Video Games. In *2024 IEEE Conference on Games (CoG)*, pages 1–8.

David Harel. 1987. Statecharts: A Visual Formalism for Complex Systems. *Science of Computer Programming*, 8(3):231–274.

Andreas Hinderks, Martin Schrepp, and Jörg Thomaschewski. 2018. A Benchmark for the Short Version of the User Experience Questionnaire:. In *Proceedings of the 14th International Conference on Web Information Systems and Technologies*, pages 373–377, Seville, Spain. SCITEPRESS - Science and Technology Publications.

Ross Mawhorter and Adam Smith. 2021. Softlock Detection for Super Metroid with Computation Tree Logic. In *Proceedings of the 16th International Conference on the Foundations of Digital Games*, pages 1–10, New York, NY, USA. Association for Computing Machinery.

Martin Schrepp, Andreas Hinderks, and Jörg Thomaschewski. 2017. Design and Evaluation of a Short Version of the User Experience Questionnaire (UEQ-S). *International Journal of Interactive Multimedia and Artificial Intelligence*, 4(6):103–108.

Adam James Summerville, Sam Snodgrass, Michael Mateas, and Santiago Ontañón. 2016. The VGLC: The Video Game Level Corpus.